Dylan Renard
EN.605.617 Introduction to GPU Programming (JHU)
Professor Chance Pascale
April 22nd 2026

OpenCL Assignment

Spectrogram Generation with CPU Baseline and OpenCL Acceleration

Figure 1: RickRoll

Overview

This project implements a spectrogram generator in C++ with both a CPU baseline and an OpenCL-accelerated execution path. The program reads `.wav` audio files, applies a Hann window across overlapping frames, computes a DFT-based power spectrogram, and writes the result as a grayscale PNG image.

The application supports both single-file and batch directory execution. In addition to the CPU baseline, the OpenCL implementation includes two DFT kernels:

- a **naive kernel** that reads frame data directly from global memory
- a **local-memory kernel** that stages frame data into OpenCL local memory before computing frequency bins

The goal of the project was to build a complete OpenCL application that demonstrates both basic and advanced OpenCL concepts on a practical workload, while also comparing CPU and GPU performance under realistic execution conditions.

Motivation

A spectrogram is a natural visualization for audio data because it shows how frequency content changes over time. This made the project a good fit for an OpenCL assignment: the workload is regular, data-parallel, visually interpretable, and large enough to show meaningful differences between CPU and GPU execution.

Rather than producing only numeric output, this project generates image artifacts that make correctness easier to inspect while also providing runtime measurements for direct performance comparison.

Data Sources

Two public audio sources were used during testing.

For batch benchmarking, the project used the GTZAN music genre dataset, specifically the `genres_original/blues` directory containing 100 audio clips. This provided a standardized 100-file workload for CPU and OpenCL batch comparison each 30 seconds in duration.

For the long-form single-file benchmark, the project used a locally prepared `rickroll_16bit.wav` file derived from a public spectrogram lyric upload of Rick Astley's *Never Gonna Give You Up*. This gave the project a much longer audio input than the GTZAN clips and made the CPU vs GPU difference much more pronounced.

For general conceptual background on spectrogram interpretation, I also referred to an introductory public video discussing how spectrograms visualize frequency over time.

System Architecture

The pipeline begins by reading a mono `.wav` file into host memory. The signal is divided into overlapping frames according to the chosen window size and hop size. Each frame is multiplied by a Hann window and then transformed into frequency-space power values using a discrete Fourier transform. The resulting spectrogram is stored as a 2D array and written to disk as a PNG image.

The CPU implementation performs this work directly on the host and serves as a correctness and performance baseline. The OpenCL implementation moves the windowing and DFT computation to the selected OpenCL device. Batch mode extends the same workflow to an entire directory of `.wav` files, enabling larger benchmarking runs on real audio collections.

Repository Structure

```
OpenCLHW/
  src/
      main.cpp
      wav_reader.cpp
      spectrogram_cpu.cpp
      spectrogram_opencl.cpp
      opencl_utils.cpp
      image_writer.cpp
  include/
      wav_reader.h
      spectrogram_cpu.h
      spectrogram_opencl.h
      opencl_utils.h
      image_writer.h
  kernels/
      spectrogram.cl
  output/
  Makefile
  Makefile.vc
  README.md
```

OpenCL Concepts Demonstrated

Basic Concepts

- platform and device discovery
- context and command queue creation
- program creation from source
- kernel creation and argument setup
- device buffer allocation
- host-to-device and device-to-host transfers
- NDRange kernel execution
- OpenCL cleanup and error handling

Advanced Concepts

- profiling-enabled command queue
- event timing for kernel-level measurement
- local memory and barrier synchronization
- vector math using `float2`
- sub-buffers for batch output partitioning
- batch processing of many files

- runtime comparison of **naive** and **local-memory** kernels
- device-aware fallback when local-memory requirements exceed capacity

Kernel Design

Window Kernel

The `window_kernel` applies a Hann window to each frame of the input signal and writes the result into an intermediate frame buffer.

Naive DFT Kernel

The naive DFT kernel reads the frame directly from global memory while each work-item computes one frequency bin.

Local-Memory DFT Kernel

The local-memory kernel cooperatively stages a frame into OpenCL local memory, synchronizes with a barrier, and then computes the DFT from the staged data. This kernel was added to demonstrate more advanced OpenCL memory usage and to benchmark whether reduced global-memory reuse would improve performance.

Build Instructions

Linux / WSL

This path uses the GNU Makefile.

```
make
```

To clean:

```
make clean
```

Windows

This path uses `nmake` and the MSVC toolchain.

```
nmake /f Makefile.vc
```

To clean:

```
nmake /f Makefile.vc clean
```

The Windows build path was especially important because it allowed the OpenCL runtime to target the NVIDIA GPU-backed implementation rather than the CPU-only OpenCL path observed earlier in WSL.

Running the Program

General Syntax

```
spectrogram <wav_file_or_directory> [cpu|gpu|both] [naive|local] [window_size] [hop_size] [num_bins]
```

Modes

- `cpu` -> host-only execution
- `gpu` -> OpenCL execution
- `both` -> run CPU and GPU for direct comparison

OpenCL Kernel Modes

- `naive`
- `local`

Example Single-File Runs

CPU:

```
./spectrogram rickroll_16bit.wav cpu 1024 512 512
```

GPU naive:

```
./spectrogram rickroll_16bit.wav gpu naive 1024 512 512
```

GPU local:

```
./spectrogram rickroll_16bit.wav gpu local 1024 512 512
```

CPU vs GPU comparison:

```
./spectrogram rickroll_16bit.wav both naive 1024 512 512
```

Example Batch Runs

Batch GPU naive:

```
./spectrogram Data/genres_original/blues gpu naive 1024 512 512
```

Batch GPU local:

```
./spectrogram Data/genres_original/blues gpu local 1024 512 512
```

Makefile Targets

GNU Make:

```
make run_cpu
make run_gpu
make run_gpu_naive
make run_gpu_local
make run_both
make batch_cpu
make batch_gpu
make batch_gpu_naive
make batch_gpu_local
make batch_both
```

Windows `nmake`:

```
nmake /f Makefile.vc run_cpu
nmake /f Makefile.vc run_gpu
nmake /f Makefile.vc run_gpu_naive
nmake /f Makefile.vc run_gpu_local
nmake /f Makefile.vc run_both
nmake /f Makefile.vc batch_cpu
nmake /f Makefile.vc batch_gpu
nmake /f Makefile.vc batch_gpu_naive
nmake /f Makefile.vc batch_gpu_local
nmake /f Makefile.vc batch_both
```

I had issue getting my WSL environment to recognize my Nvidia GPU so adding a windows specific Makefile was implemented to allow for practical speedup testing beyond CPU only benchmarks.

Output

The program writes PNG spectrogram images to the `output/` directory.

Example single-file outputs: - `output/output_cpu.png` - `output/output_gpu_naive.png` - `output/output_gpu_local.png`

Example batch outputs: - `output/blues.00000_gpu_naive.png` - `output/blues.00000_gpu_local.png`

These image outputs serve as both proof of execution and an easy visual check that the computation completed correctly.

Performance Results

1. CPU Batch Baseline on 100 Blues Clips

A batch run over the 100 blues clips established the host-side baseline.

Observed CPU batch timings: - Files processed: **100** - WAV read time: ~**6.5–10.7 s** - Spectrogram time: ~**958.9–978.9 s** - PNG write time: ~**8.0–8.6 s** - Total runtime: ~**974.2–997.9 s**

These runs show that spectrogram generation dominates runtime on the CPU.

```
CPU Batch Runtime Summary
-------------------------
Files processed:       100
WAV read time:         10723 ms
Spectrogram time:      978918 ms
PNG write time:        8031 ms
Total runtime:         997873 ms
```

Figure 2: CPU batch runtime on 100 blues clips

```
CPU Batch Runtime Summary
-------------------------
Files processed:       100
WAV read time:         6510 ms
Spectrogram time:      958906 ms
PNG write time:        8622 ms
Total runtime:         974243 ms
```

Figure 3: CPU batch runtime alternate view

2. OpenCL in WSL: CPU-Backed Device Selection

During development, the OpenCL path was first tested in WSL. In that environment, OpenCL selected a CPU-backed Intel device rather than the NVIDIA GPU:

- `OpenCL device: GenuineIntel - cpu-haswell-13th Gen Intel(R) Core(TM) i7-13620H`

This caused the OpenCL path to perform far below the final Windows-native GPU runs.

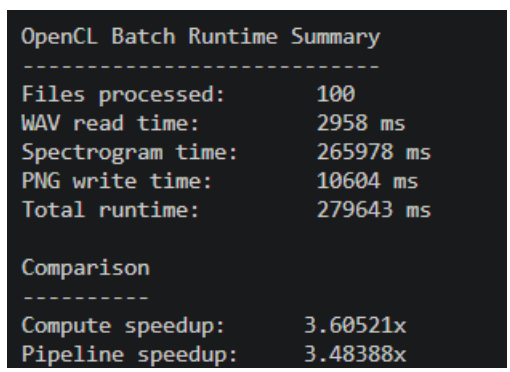
WSL OpenCL local-memory batch run

- Files processed: **100**
- WAV read time: **6496 ms**
- Spectrogram time: **265033 ms**
- PNG write time: **9904 ms**
- Total runtime: **281665 ms**
- Summed GPU kernel ms: **262665 ms**

WSL OpenCL naive batch run

- Files processed: **100**
- WAV read time: **2577 ms**
- Spectrogram time: **268658 ms**
- PNG write time: **10705 ms**
- Total runtime: **282196 ms**
- Summed GPU kernel ms: **267904 ms**

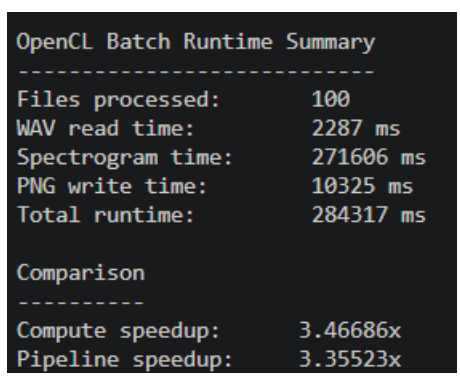
The local-memory kernel was only slightly faster than naive in this CPU-backed OpenCL environment, and overall speedup relative to the CPU baseline was only around **3.5x**. This turned out to be more of an OpenCL environment/device selection issue than a kernel-design issue.



```
OpenCL Batch Runtime Summary
----------------------------
Files processed:       100
WAV read time:         2958 ms
Spectrogram time:      265978 ms
PNG write time:        10604 ms
Total runtime:         279643 ms

Comparison
----------
Compute speedup:      3.60521x
Pipeline speedup:     3.48388x
```

Figure 4: WSL OpenCL batch runtime advanced view



```
OpenCL Batch Runtime Summary
----------------------------
Files processed:       100
WAV read time:         2287 ms
Spectrogram time:      271606 ms
PNG write time:        10325 ms
Total runtime:         284317 ms

Comparison
----------
Compute speedup:      3.46686x
Pipeline speedup:     3.35523x
```

Figure 5: WSL OpenCL batch runtime second advanced view

3. Native Windows OpenCL on the NVIDIA GPU

After building and running the project natively on Windows, OpenCL correctly selected the NVIDIA GPU:

- `OpenCL device: NVIDIA Corporation - NVIDIA GeForce RTX 4060 Laptop GPU`

```
OpenCL Batch Runtime Summary (local)
-----------------------------------
Files processed:       100
WAV read time:         6496 ms
Spectrogram time:      265033 ms
PNG write time:        9904 ms
Total runtime:         281665 ms
Summed GPU kernel ms:  262665 ms
```

Figure 6: WSL local-memory OpenCL runtime

```
OpenCL device: GenuineIntel - cpu-haswell-13th Gen Intel(R) Core(TM) i7-13620H
OpenCL batch window kernel total: 404.711 ms
OpenCL batch DFT kernel total (local): 262311 ms
```

Figure 7: WSL local-memory kernel runtimes on CPU-backed OpenCL

```
OpenCL Batch Runtime Summary (naive)
-----------------------------------
Files processed:       100
WAV read time:         2577 ms
Spectrogram time:      268658 ms
PNG write time:        10705 ms
Total runtime:         282196 ms
Summed GPU kernel ms:  267904 ms
```

Figure 8: WSL naive OpenCL batch runtime

```
./bin/spectrogram "Data/genres_original/blues" gpu naive "1024" "512" "512"
OpenCL device: GenuineIntel - cpu-haswell-13th Gen Intel(R) Core(TM) i7-13620H
OpenCL batch window kernel total: 406.893 ms
OpenCL batch DFT kernel total (naive): 267556 ms
```

Figure 9: WSL naive OpenCL batch kernel runtime

This changed performance dramatically.

Windows OpenCL batch naive

- Files processed: **100**
- WAV read time: **320 ms**
- Spectrogram time: **2093 ms**
- PNG write time: **7694 ms**
- Total runtime: **10748 ms**
- Summed GPU kernel ms: **1339 ms**

Windows OpenCL batch local

- Files processed: **100**
- WAV read time: **334 ms**
- Spectrogram time: **3098 ms**
- PNG write time: **7736 ms**
- Total runtime: **11743 ms**
- Summed GPU kernel ms: **1347 ms**

The improvement over the WSL CPU-backed OpenCL path was dramatic. Batch spectrogram time dropped from roughly **265–269 seconds** to roughly **2.1–3.1 seconds**.

Interestingly, the **naive kernel outperformed the local-memory kernel** on the final Windows GPU-backed run. Their kernel times were nearly identical, but the naive path produced the better overall spectrogram time. For this direct-DFT workload and launch configuration, the overhead of local-memory staging and synchronization appears to outweigh the benefit of reduced global-memory access.

```
OpenCL Batch Runtime Summary
----------------------------
Files processed:       100
WAV read time:         3580 ms
Spectrogram time:      478189 ms
PNG write time:        11199 ms
Total runtime:         493080 ms
```

Figure 10: Windows OpenCL runtime on 100 blues songs

```
OpenCL Batch Runtime Summary (naive)
------------------------------------
Files processed:       100
WAV read time:         320 ms
Spectrogram time:      2093 ms
PNG write time:        7694 ms
Total runtime:         10748 ms
Summed GPU kernel ms:  1339 ms
```

Figure 11: Windows OpenCL naive batch runtime

4. Single Long-Form Audio Benchmark: `rickroll_16bit.wav`

The strongest benchmark in the project came from comparing CPU and GPU on a long real audio file.

Input characteristics: - file: `rickroll_16bit.wav` - duration: **212.01 seconds** - sample rate: **44100 Hz** - channels: **1** - frames: **18260** - bins: **512**

```
C:\Users\drena\OneDrive\Desktop\CodingRepos\GPU-Programming\OpenCLHW>spectrogram.exe "Data\genres_original\blues" gpu naive 102
4 512 512
OpenCL device: NVIDIA Corporation - NVIDIA GeForce RTX 4060 Laptop GPU
OpenCL batch window kernel total: 3.49053 ms
OpenCL batch DFT kernel total (naive): 1383.12 ms
```

Figure 12: Windows OpenCL naive kernel runtime

```
OpenCL Batch Runtime Summary (local)
------------------------------------
Files processed:       100
WAV read time:         334 ms
Spectrogram time:      3098 ms
PNG write time:        7736 ms
Total runtime:         11743 ms
Summed GPU kernel ms:  1347 ms
```

Figure 13: Windows OpenCL local-memory runtime

CPU result

- WAV read time: **63 ms**
- Spectrogram time: **197377 ms**
- PNG write time: **844 ms**
- Total runtime: **198291 ms**

OpenCL GPU naive result

- OpenCL device: **NVIDIA GeForce RTX 4060 Laptop GPU**
- Window kernel: **0.80384 ms**
- DFT kernel: **235.269 ms**
- WAV read time: **66 ms**
- Spectrogram time: **595 ms**
- PNG write time: **1104 ms**
- Total runtime: **1784 ms**

Speedup

- **Compute speedup: 331.726x**
- **Pipeline speedup: 111.15x**

This was the clearest demonstration of the project's value. The CPU baseline required over **197 seconds** of spectrogram computation, while the OpenCL GPU path completed the same stage in under **0.6 seconds**.

Additional single-file naive vs local comparison

On the same `rickroll_16bit.wav` input:

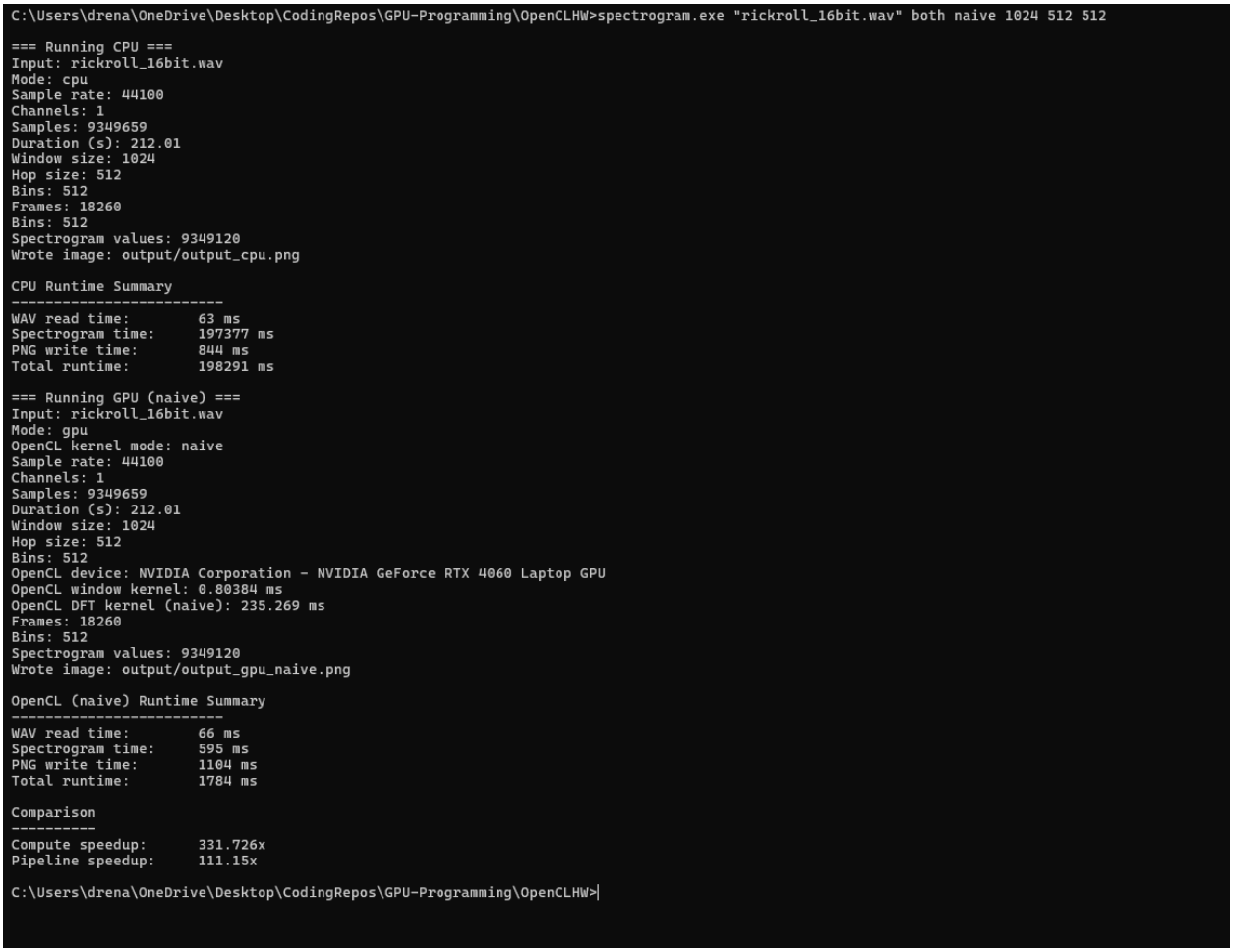
GPU naive - WAV read time: **84 ms** - Spectrogram time: **532 ms** - PNG write time: **904 ms** - Total runtime: **1529 ms** - DFT kernel time: **232.617 ms**

```
C:\Users\drena\OneDrive\Desktop\CodingRepos\GPU-Programming\OpenCLHW>spectrogram.exe "Data\genres_original\blues" gpu local 102
4 512 512
OpenCL device: NVIDIA Corporation - NVIDIA GeForce RTX 4060 Laptop GPU
OpenCL batch window kernel total: 3.99558 ms
OpenCL batch DFT kernel total (local): 1383.31 ms
```

Figure 14: Windows OpenCL local kernel runtime

GPU local - WAV read time: **51 ms** - Spectrogram time: **548 ms** - PNG write time: **917 ms** - Total runtime: **1523 ms** - DFT kernel time: **216.626 ms**

These single-file runs show that naive and local were close on this workload, with local slightly lowering the DFT kernel time in one run but not producing a clear overall end-to-end advantage.



```
C:\Users\drena\OneDrive\Desktop\CodingRepos\GPU-Programming\OpenCLHW>spectrogram.exe "rickroll_16bit.wav" both naive 1024 512 512

=== Running CPU ===
Input: rickroll_16bit.wav
Mode: cpu
Sample rate: 44100
Channels: 1
Samples: 9349659
Duration (s): 212.01
Window size: 1024
Hop size: 512
Bins: 512
Frames: 18260
Bins: 512
Spectrogram values: 9349120
Wrote image: output/output_cpu.png

CPU Runtime Summary
-------------------------
WAV read time:        63 ms
Spectrogram time:     197377 ms
PNG write time:       844 ms
Total runtime:        198291 ms

=== Running GPU (naive) ===
Input: rickroll_16bit.wav
Mode: gpu
OpenCL kernel mode: naive
Sample rate: 44100
Channels: 1
Samples: 9349659
Duration (s): 212.01
Window size: 1024
Hop size: 512
Bins: 512
OpenCL device: NVIDIA Corporation - NVIDIA GeForce RTX 4060 Laptop GPU
OpenCL window kernel: 0.80384 ms
OpenCL DFT kernel (naive): 235.269 ms
Frames: 18260
Bins: 512
Spectrogram values: 9349120
Wrote image: output/output_gpu_naive.png

OpenCL (naive) Runtime Summary
------------------------
WAV read time:        66 ms
Spectrogram time:     595 ms
PNG write time:       1104 ms
Total runtime:        1784 ms

Comparison
----------
Compute speedup:      331.726x
Pipeline speedup:     111.15x

C:\Users\drena\OneDrive\Desktop\CodingRepos\GPU-Programming\OpenCLHW>
```

Figure 15: GPU naive vs CPU baseline on RickRoll

Interpretation

Several important lessons emerged from the experiments.

First, OpenCL device selection mattered enormously. The same OpenCL code produced radically different performance depending on whether it executed through a CPU-backed OpenCL runtime in WSL or a GPU-backed OpenCL runtime on native Windows.

Second, the project clearly demonstrated the value of GPU acceleration for spectrogram generation. On the long-form Rick Astley benchmark, the OpenCL GPU implementation reduced spectrogram compute time from **197.4 seconds** to under **0.6 seconds**, producing a **331.7x compute speedup**.

Third, the advanced local-memory kernel was still valuable even though it was not the fastest final kernel. Implementing it required correct use of local memory, barriers, cooperative loading, vector types, and more careful host-side kernel argument management. Benchmarking it against the naive kernel created a real optimization study rather than a superficial feature add-on.

```
C:\Users\drena\OneDrive\Desktop\CodingRepos\GPU-Programming\OpenCLHW>spect
2
Input: rickroll_16bit.wav
Mode: gpu
OpenCL kernel mode: naive
Sample rate: 44100
Channels: 1
Samples: 9349659
Duration (s): 212.01
Window size: 1024
Hop size: 512
Bins: 512
OpenCL device: NVIDIA Corporation - NVIDIA GeForce RTX 4060 Laptop GPU
OpenCL window kernel: 0.806912 ms
OpenCL DFT kernel (naive): 232.617 ms
Frames: 18260
Bins: 512
Spectrogram values: 9349120
Wrote image: output/output_gpu_naive.png

OpenCL (naive) Runtime Summary
--------------------------
WAV read time:        84 ms
Spectrogram time:     532 ms
PNG write time:       904 ms
Total runtime:        1529 ms
```

Figure 16: GPU OpenCL RickRoll naive runtime

```
C:\Users\drena\OneDrive\Desktop\CodingRepos\GPU-Programming\OpenCLHW>spectrogra
2
Input: rickroll_16bit.wav
Mode: gpu
OpenCL kernel mode: local
Sample rate: 44100
Channels: 1
Samples: 9349659
Duration (s): 212.01
Window size: 1024
Hop size: 512
Bins: 512
OpenCL device: NVIDIA Corporation - NVIDIA GeForce RTX 4060 Laptop GPU
OpenCL window kernel: 1.29434 ms
OpenCL DFT kernel (local): 216.626 ms
Frames: 18260
Bins: 512
Spectrogram values: 9349120
Wrote image: output/output_gpu_local.png

OpenCL (local) Runtime Summary
------------------------
WAV read time:        51 ms
Spectrogram time:     548 ms
PNG write time:       917 ms
Total runtime:        1523 ms
```

Figure 17: GPU OpenCL RickRoll local runtime

Finally, once true GPU execution was achieved, **PNG writing became a large fraction of total runtime** in batch mode. This indicates that OpenCL compute became efficient enough that output generation and file handling started to dominate the overall pipeline.

Challenges and Triumphs

A major challenge in this project was that the earliest OpenCL runs in WSL did not use the intended NVIDIA GPU. Instead, OpenCL selected a CPU-backed Intel device, leading to runtimes that made the program appear far slower than it actually was. Diagnosing this required checking device strings, comparing runtime behavior across environments, and eventually moving to a native Windows build path.

Another challenge was implementing and wiring both the naive and local-memory kernels correctly. The two kernels required different argument layouts and different host-side handling. Batch mode added further complexity through sub-buffer creation, event management, and multi-file scheduling.

The main triumph of the project was the final native Windows GPU-backed run. Once the program targeted the RTX 4060 correctly, the OpenCL version achieved massive speedups over the CPU baseline and produced the kind of result that justifies GPU acceleration for this workload.

Conclusion

This project demonstrates that OpenCL can provide substantial acceleration for spectrogram generation whether it be a 2-3x speedup on the same hardware or a 300x speedup if employing the GPU. Demonstrating the cross-platform benefit OpenCL has across different hardware.

The final Windows-native OpenCL implementation running on an NVIDIA GeForce RTX 4060 Laptop GPU achieved strong performance on both batch and single-file audio workloads, including a **331.7x compute speedup** over the CPU baseline on a 212-second WAV file.

At the same time, the project reinforced a more important GPU programming lesson: advanced features do not automatically guarantee better performance. Although a local-memory DFT kernel was implemented correctly using cooperative loading and barrier synchronization, it did not consistently outperform the naive kernel on the final GPU-backed runs. This result was still valuable, because it showed that optimization choices must be validated experimentally rather than assumed.

Overall, this project moved beyond a toy OpenCL example by combining CPU and GPU implementations, single-file and batch execution, multiple build paths, kernel profiling, environment-aware analysis, and direct naive-vs-local kernel comparison in one complete application.

Sources

The following public sources were used during development, testing, and interpretation of results.

1. Andrada Olteanu. *GTZAN Dataset - Music Genre Classification*. Kaggle. Used for batch benchmarking, especially the `genres_original/blues` subset containing 100 audio clips.
   https://www.kaggle.com/datasets/andradaolteanu/gtzan-dataset-music-genre-classification
2. *Rick Astley - Never Gonna Give You Up* spectrogram-style YouTube source. Used as the basis for preparing the long-form `rickroll_16bit.wav` benchmark input used in single-file CPU vs GPU testing.
   https://www.youtube.com/watch?v=y24ZI-8Q9mk
3. Introductory spectrogram explanation video. Used for general conceptual reference on how spectrograms represent frequency over time.
   https://www.youtube.com/watch?v=_FatxGN3vAM